

Pigs: Evaluating Phased Orchestration for LLM API Requests

Anonymous

Abstract

We present an experimental evaluation of Pigs, a phased orchestration middleware that wraps LLM API requests with a Pre→Executor→Post state machine using text control markers (PIGEND/PIGFAIL). While the companion system design paper [Anonymous, 2026] describes the architecture, this paper focuses on *whether* phased orchestration improves task completion quality. We design four experiments: (1) an ablation study comparing full orchestration against no orchestration and partial configurations; (2) a control marker effectiveness study isolating the contribution of PIGFAIL-based failure recovery; (3) a three-protocol consistency evaluation across OpenAI Chat, Anthropic Messages, and OpenAI Responses; and (4) a cost-benefit analysis quantifying the overhead of orchestration. We use public benchmarks including SWE-bench, GAIA, and AgentBoard. **Experimental results are pending and will be filled upon completion of the evaluation.**

1 Introduction

The companion system design paper [Anonymous, 2026] introduced Pigs, a phased orchestration middleware for LLM API requests. Pigs wraps each request with a Pre→Executor→Post state machine, using text control markers (PIGEND/PIGFAIL) for routing. The system preserves protocol-native HTTP requests across three API protocols and supports external tool execution through bounded in-memory continuation.

While the system design is described in detail, a key question remains: *does phased orchestration actually improve task completion quality?* This paper addresses that question through a systematic experimental evaluation.

We make the following contributions:

1. An **ablation study** isolating the contribution of each phase (Pre, Executor, Post) to overall task performance.
2. A **control marker effectiveness study** showing whether PIGFAIL-based failure recovery improves outcomes.
3. A **three-protocol consistency evaluation** verifying that the system produces consistent results across OpenAI Chat, Anthropic Messages, and OpenAI Responses.
4. A **cost-benefit analysis** quantifying the token, latency, and API call overhead of orchestration.

All experiments use public benchmarks (SWE-bench, GAIA, AgentBoard) and require only API access—no GPU resources.

2 Experimental Method

2.1 System Configuration

All experiments are conducted through the Pigs HTTP API. The system is configured with:

- Default orchestration limits: `max_post_iterations=3`, `max_pre_replans=2`
- Continuation TTL: 30 minutes, capacity: 256
- Language: Chinese (zh) for phase prompts
- Thinking effort: xhigh (OpenAI) / max (Anthropic)

2.2 Ablation Conditions

For Experiment 1, we define four conditions:

- **Condition A (Baseline):** Direct API call without orchestration (non-`-pig` model, passthrough).
- **Condition B (Full):** Complete Pre→Executor→Post orchestration (`-pig` model).
- **Condition C (Pre-only):** Only the Pre phase (planning without execution verification).
- **Condition D (Executor+Post):** Skip Pre, directly execute and review.

2.3 Metrics

- **Task completion rate:** percentage of tasks successfully completed (benchmark-specific definition).
- **Average turns:** mean number of LLM API calls per task.
- **Token consumption:** total input + output tokens per task.
- **Latency:** wall-clock time from request to final response.
- **Error recovery rate:** percentage of tasks where PIGFAIL triggered a successful replan.

2.4 Benchmarks

Benchmark	Domain	Tasks	Public
SWE-bench Lite [Jimenez et al., 2024]	Software engineering	300	✓
GAIA [Mialon et al., 2023]	General AI assistant	300	✓
AgentBoard [Ma et al., 2024]	Multi-turn agent	9 tasks	✓

3 Experiments

3.1 Experiment 1: Ablation Study

Setup. We evaluate four conditions (A: baseline, B: full orchestration, C: Pre-only, D: Executor+Post) on SWE-bench Lite (300 tasks) and GAIA (300 tasks). Each task is sent as a `-pig` model request (Condition B/C/D) or a passthrough request (Condition A) to the Pigs API.

Procedure. For each task:

1. Send the task prompt to the Pigs API endpoint.
2. If tool calls are returned, execute them locally and send results back.
3. Repeat until the API returns a final response (no tool calls).
4. Evaluate the final response against the benchmark’s ground truth.

Hypothesis. Full orchestration (B) outperforms baseline (A) on task completion rate, with the Pre phase contributing planning and the Post phase contributing verification.

3.2 Experiment 2: Control Marker Effectiveness

Setup. We compare two configurations on GAIA: (A) with PIGFAIL enabled (failure triggers replan), (B) with PIGFAIL disabled (failure treated as completion).

Procedure. We count: (1) tasks where PIGFAIL was emitted, (2) tasks where replan led to successful completion, (3) tasks that failed despite replan.

Hypothesis. PIGFAIL-based replan improves outcomes on tasks where the initial approach was flawed.

3.3 Experiment 3: Three-Protocol Consistency

Setup. We send the same set of 100 tasks through three API protocols: OpenAI Chat (`/chat/completions`), Anthropic Messages (`/v1/messages`), and OpenAI Responses (`/responses`).

Procedure. For each task and protocol, we record: (1) whether the task completed, (2) the visible text output, (3) any tool calls made. We measure consistency as the percentage of tasks where all three protocols agree on completion status.

Hypothesis. The protocol-native request preservation ensures consistent behavior across all three protocols.

3.4 Experiment 4: Cost-Benefit Analysis

Setup. On SWE-bench Lite, we measure per-task: (1) total input tokens, (2) total output tokens, (3) number of API calls, (4) wall-clock latency, (5) task completion.

Procedure. We compare Condition A (baseline) vs. Condition B (full orchestration) and compute the overhead ratio for each metric.

Hypothesis. The token overhead of orchestration (3 phases vs. 1) is justified by improved completion rate.

4 Results

[All experimental results are pending. The tables below will be filled upon completion of the evaluation.]

4.1 Experiment 1: Ablation Results

Benchmark	Condition	Completion Rate (%)	Avg. Turns	Tokens (k)	Latency (s)
4*SWE-bench Lite	A (Baseline)	—	—	—	—
	B (Full)	—	—	—	—
	C (Pre-only)	—	—	—	—
	D (Exec+Post)	—	—	—	—
4*GAIA	A (Baseline)	—	—	—	—
	B (Full)	—	—	—	—
	C (Pre-only)	—	—	—	—
	D (Exec+Post)	—	—	—	—

4.2 Experiment 2: Control Marker Results

Configuration	PIGFAIL emitted (%)	Replan succeeded (%)	Overall completion (%)
A (With PIGFAIL)	—	—	—
B (Without PIGFAIL)	N/A	N/A	—

4.3 Experiment 3: Protocol Consistency

Protocol	Completion (%)	Tool calls	Text agreement	Consistency
OpenAI Chat	—	—	—	—
Anthropic Messages	—	—	—	—
OpenAI Responses	—	—	—	—

4.4 Experiment 4: Cost-Benefit

Condition	Input tokens (k)	Output tokens (k)	API calls	Completion (%)
A (Baseline)	—	—	—	—
B (Full)	—	—	—	—
Overhead	—×	—×	—×	—pp

5 Related Work

This paper builds on the system design described in the companion paper [Anonymous, 2026]. The experimental methodology draws from agent evaluation benchmarks: **AgentBench** [Liu et al., 2024] for

multi-environment evaluation, **SWE-bench** [Jimenez et al., 2024] for software engineering, **GAIA** [Mialon et al., 2023] for general AI assistant evaluation, and **AgentBoard** [Ma et al., 2024] for fine-grained progress analysis.

The ablation methodology is inspired by component-level analysis in multi-stage reasoning systems. Yao et al. [2023] showed that interleaved reasoning and acting improves over pure reasoning; Shinn et al. [2023] demonstrated that reflection improves over no reflection; Madaan et al. [2023] showed iterative refinement improves output quality. Our ablation extends this to a three-phase model, isolating Pre (planning) and Post (verification) independently.

The cost-benefit analysis follows the methodology of Kaddour et al. [2023], who discuss the trade-offs of LLM complexity. The three-protocol consistency evaluation is novel—no prior work systematically compares agent behavior across different API protocols.

Huang et al. [2024] found that LLMs cannot reliably self-correct reasoning without external feedback, which motivates our Post phase as a structured review step rather than relying on intrinsic self-correction.

6 Conclusion

We designed an experimental evaluation framework for Pigs phased orchestration, comprising four experiments: ablation, control marker effectiveness, three-protocol consistency, and cost-benefit analysis. All experiments use public benchmarks and require only API access. Experimental results are pending and will be reported upon completion of the evaluation.

Future work includes: (1) multi-model orchestration evaluation (different models for different phases), (2) extended benchmarks (AgentBoard, ToolBench), (3) formal verification of state machine properties, and (4) security analysis against prompt injection attacks [Greshake et al., 2023].

References

- Anonymous. Pigs: Protocol-native phased orchestration for llm api requests, 2026. Companion system design paper.
- Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. *arXiv preprint arXiv:2302.12173*, 2023.
- Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large language models cannot self-correct reasoning yet. In *International Conference on Learning Representations (ICLR)*, 2024.
- Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations (ICLR)*, 2024.
- Jean Kaddour, Joshua Harris, Maximilian Mozes, Herbie Bradley, Roberta Raileanu, and Robert McHardy. Challenges and applications of large language models. *arXiv preprint arXiv:2307.10169*, 2023.
- Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. Agentbench: Evaluating llms as agents. In *International Conference on Learning Representations (ICLR)*, 2024.
- Chang Ma, Jun Ma, Kaixin Liu, Junzhe Li, Zhihao Mei, Guang Zhang, Zhan Wang, Yiqun Gu, Zhibo Xu, Zhijian Li, et al. Agentboard: An analytical evaluation board of multi-turn llm agents. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. Gaia: A benchmark for general ai assistants. *arXiv preprint arXiv:2311.12983*, 2023.

Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.